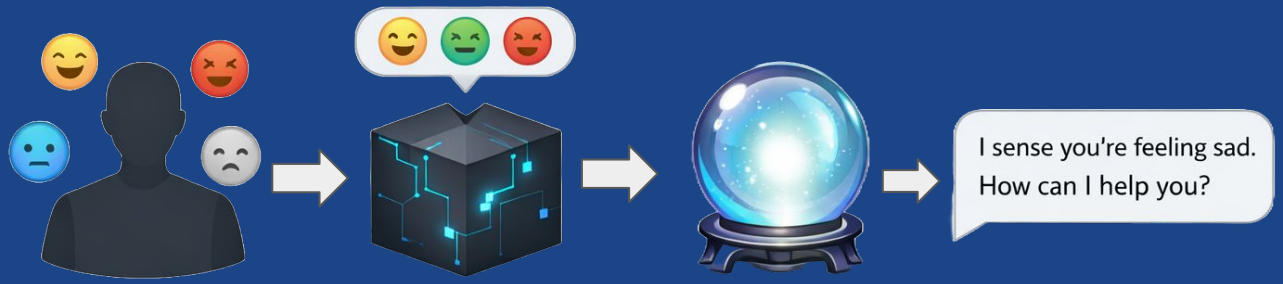
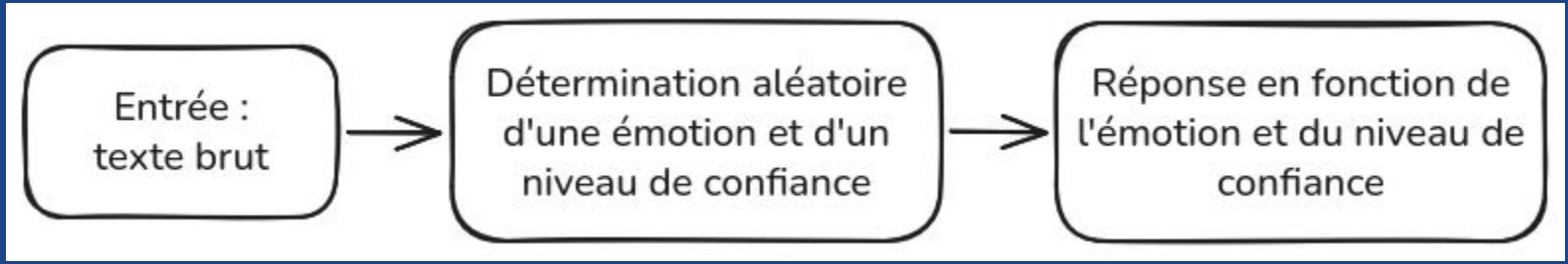
Projet Oracle
Conduite de projet

Elliot DURAND
Yanis CHAIBI
M1 Génie Logiciel

Contexte



Fonctionnement du premier code

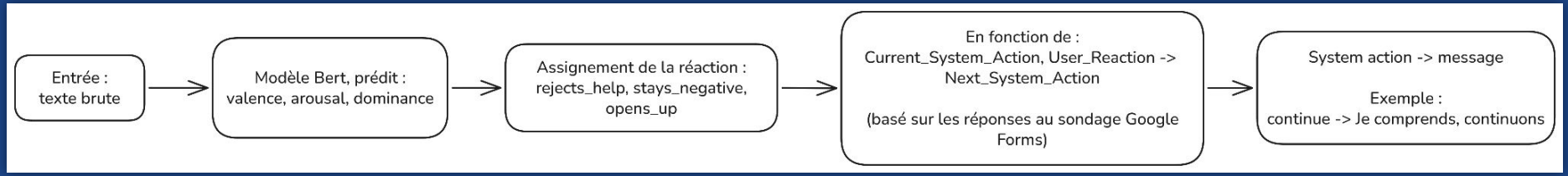


Code de base

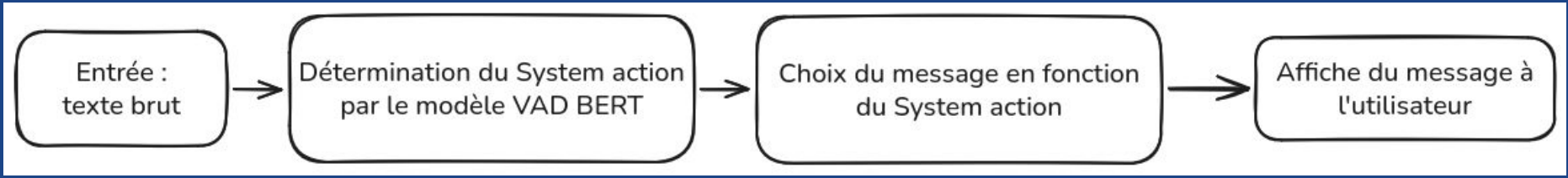
- Mise en place d'un "Oracle" temporaire random facilement remplaçable
- Variable destiné à l'input utilisateur prévue
- Code facilement modulable

```python
class Oracle:
    def __init__(self):
        self.emotions = ["joy", "sadness", "anger", "fear", "disgust", "surprise"]
        self.confidences = ["low", "medium", "high"]

    def detect(self, text: str) -> dict:
        return {
            "emotion": random.choice(self.emotions),
            "confidence": random.choice(self.confidences)
        }

class DialogueManager:
    def __init__(self):
        self.messages = {
            "continue": "Je comprends, continuons",
            "ask_clarification": "Pouvez-vous m'en dire plus ?",
            "slow_down": "Prenons un instant pour réfléchir",
            "offer_support": "Je suis là pour vous soutenir",
            "de_escalate": "Restons calme et analysons la situation",
            "suggest_pause": "Vous devriez prendre une pause"
        }

    def get_reply(self, analysis: dict) -> dict:
        action = random.choice(list(self.messages.keys()))

        return {
            "action": action,
            "message": self.messages[action],
            "detected_emotion": analysis["emotion"],
            "confidence": analysis["confidence"]
        }

class Chatbot:
    def __init__(self):
        self.oracle = Oracle()
        self.dialogue = DialogueManager()

    def process(self, user_input: str):
        analysis = self.oracle.detect(user_input)

        response = self.dialogue.get_reply(analysis)

        return response

if __name__ == "__main__":
    # Input et appel à process()
```

Fonctionnement du modèle Bert



Fonctionnement du deuxième code



Code avec intégration de l'IA

- Découverte de l'Oracle donné utilisant le modèle VAD
- Intégration de la pipeline de l'Oracle à la place du random (process_json_request)

```python
class Oracle:
    def __init__(self):
        pass

    def detect(self, text: str, current_action: str) -> dict:
        payload = json.dumps({
            "user_text": text,
            "current_system_action": current_action
        })

        return json.loads(process_json_request(payload))

class DialogueManager:
    def __init__(self):
        self.messages = {
            "continue": "Je comprends, continuons",
            "ask_clarification": "Pouvez-vous m'en dire plus ?",
            "slow_down": "Prenons un instant pour réfléchir",
            "offer_support": "Je suis là pour vous soutenir",
            "de_escalate": "Restons calme et analysons la situation",
            "suggest_pause": "Vous devriez prendre une pause",
            "acknowledge": "Je comprends ce que vous ressentez",
            "encourage": "C'est très positif, je vous écoute"
        }

    def get_reply(self, analysis: dict) -> dict:
        action = analysis["next_system_action"]

        return {
            "action": action,
            "message": self.messages.get(action, "Pouvez-vous m'en dire plus ?"),
            "detected_emotion": analysis["mapped_vad_state"],
            "vad_scores": analysis["vad_scores"]
        }

class Chatbot:
    def __init__(self):
        self.oracle = Oracle()
        self.dialogue = DialogueManager()
        self.current_action = "ask_clarification"

    def process(self, user_input: str):
        analysis = self.oracle.detect(user_input, self.current_action)

        response = self.dialogue.get_reply(analysis)

        self.current_action = response["action"]

        return response

if __name__ == "__main__":
    # Input et appel à process()
```

Évolution du code

Original

```diff
class Oracle:
    def __init__(self):
-       self.emotions = ["joy", "sadness", "anger", "fear", "disgust", "surprise"]
-       self.confidences = ["low", "medium", "high"]

-   def detect(self, text: str) -> dict:
-       return {
-           "emotion": random.choice(self.emotions),
-           "confidence": random.choice(self.confidences)
-       }

class DialogueManager:
    def __init__(self):
        self.messages = {
Expand 3 lines ...
            "offer_support": "Je suis là pour vous soutenir",
            "de_escalate": "Restons calme et analysons la situation",
            "suggest_pause": "Vous devriez prendre une pause",

        }

    def get_reply(self, analysis: dict) -> dict:
-       action = random.choice(list(self.messages.keys()))

        return {
            "action": action,
-           "message": self.messages[action],
-           "detected_emotion": analysis["emotion"],
-           "confidence": analysis["confidence"]
        }

class Chatbot:
    def __init__(self):
        self.oracle = Oracle()
        self.dialogue = DialogueManager()

    def process(self, user_input: str):
-       analysis = self.oracle.detect(user_input)

        response = self.dialogue.get_reply(analysis)

        return response

if __name__ == "__main__":
    bot = Chatbot()

    while True:
        entry = input("Utilisateur (tapez 'quit' pour sortir) : ")
        if (entry.lower() == "quit"):
            print("Au revoir !")
            break

        result = bot.process(entry)
        print(f"Chatbot: {json.dumps(result, indent=4)}")
```

Changed

```diff
class Oracle:
    def __init__(self):
+       pass

+   def detect(self, text: str, current_action: str) -> dict:
+       payload = json.dumps({
+           "user_text": text,
+           "current_system_action": current_action
+       })

+       return json.loads(process_json_request(payload))
+
class DialogueManager:
    def __init__(self):
        self.messages = {
            "offer_support": "Je suis là pour vous soutenir",
            "de_escalate": "Restons calme et analysons la situation",
            "suggest_pause": "Vous devriez prendre une pause",
+           "acknowledge": "Je comprends ce que vous ressentez",
+           "encourage": "C'est très positif, je vous écoute"
        }

    def get_reply(self, analysis: dict) -> dict:
+       action = analysis["next_system_action"]

        return {
            "action": action,
+           "message": self.messages.get(action, "Pouvez-vous m'en dire plus ?"),
+           "detected_emotion": analysis["mapped_vad_state"],
+           "vad_scores": analysis["vad_scores"]
        }

class Chatbot:
    def __init__(self):
        self.oracle = Oracle()
        self.dialogue = DialogueManager()
+       self.current_action = "ask_clarification"

    def process(self, user_input: str):
+       analysis = self.oracle.detect(user_input, self.current_action)

        response = self.dialogue.get_reply(analysis)

+       self.current_action = response["action"]
+
        return response

if __name__ == "__main__":
    bot = Chatbot()

    while True:
        entry = input("Utilisateur (tapez 'quit' pour sortir) : ")
        if (entry.lower() == "quit"):
            print("Au revoir !")
            break

        result = bot.process(entry)
        print(f"Chatbot: {json.dumps(result, indent=4)}")
```

Démo

```
Utilisateur (tapez 'quit' pour sortir) : i am feeling good

🤖 Chatbot
➡️Action: encourage
💬 Message: C'est très positif, je vous écoute
😊 Émotion détectée: opens_up
📊 Scores VAD: {"valence": 4.25, "arousal": 3.41, "dominance": 3.37}

Utilisateur (tapez 'quit' pour sortir) : i am sad now

🤖 Chatbot
➡️Action: offer_support
💬 Message: Je suis là pour vous soutenir
😊 Émotion détectée: stays_negative
📊 Scores VAD: {"valence": 1.59, "arousal": 3.32, "dominance": 2.59}
```

Conclusion